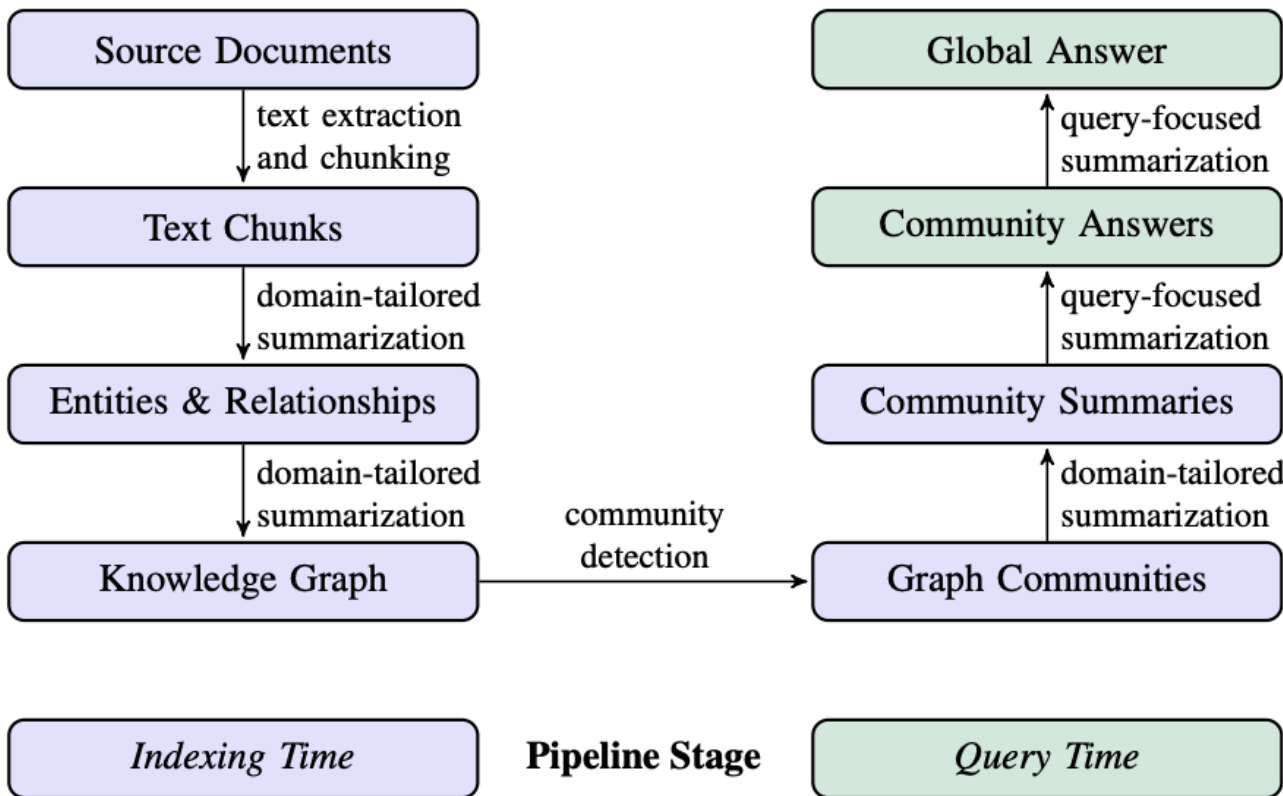


融合Neo4j与LlamaIndex：深度解析DRIFT搜索的实现与创新

By [NEXTECH](#) [Save It](#)

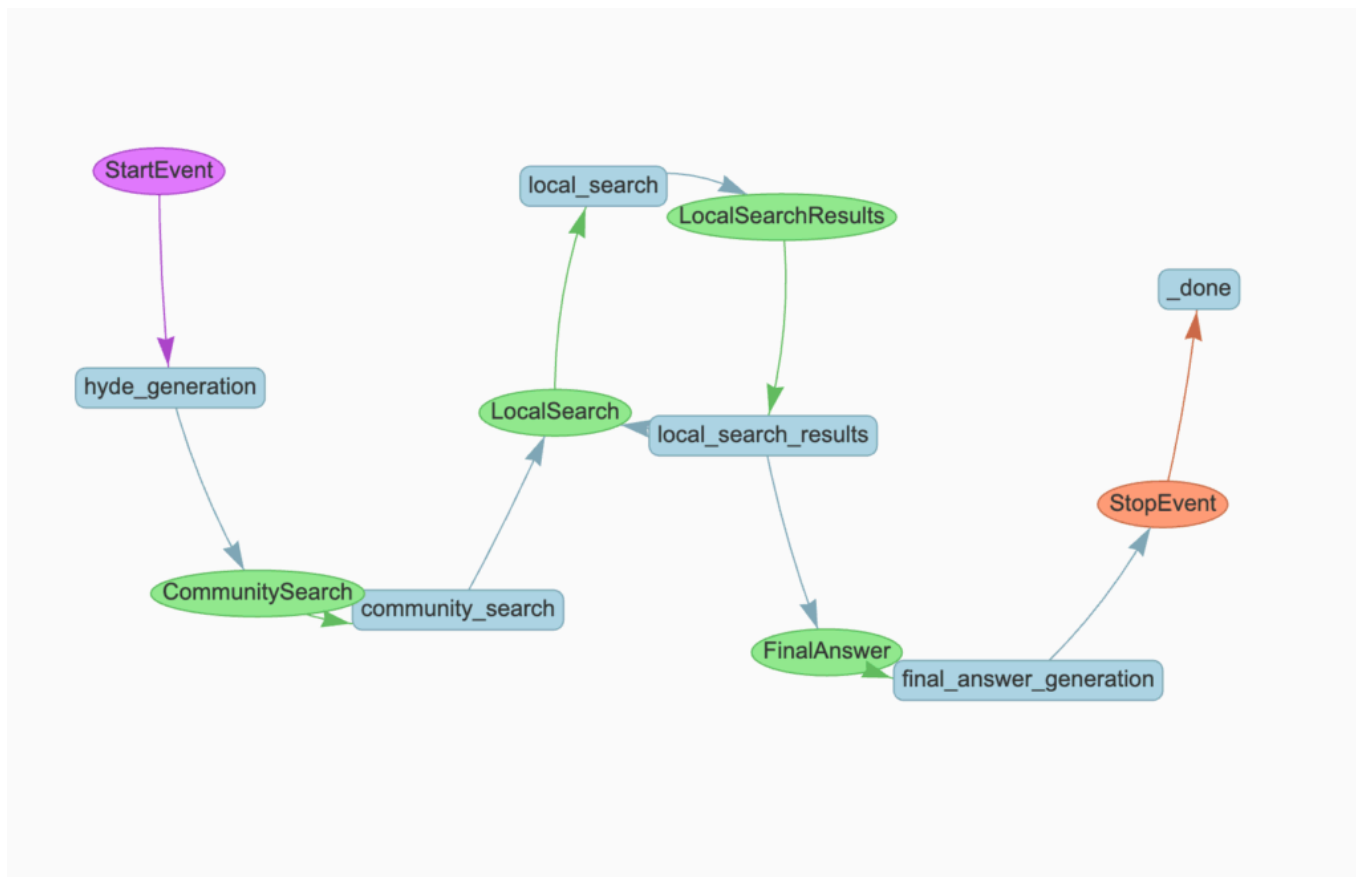
Last Updated: 2025年10月23日 上午6:24

微软的GraphRAG实现是首批开创性的系统之一，引入了诸多创新特性。它巧妙地将索引阶段（实体、关系和分层社区的提取与摘要）与先进的查询时能力相结合。这种方法使得系统能够超越传统RAG系统仅限于文档检索的局限，通过利用预先计算的实体、关系和社区摘要，回答广泛而主题性的问题。



微软GraphRAG管道示意图。图片来源：[Edge et al., 2024]，遵循CC BY 4.0许可。

鉴于之前的博客文章（[此处](#)和[此处](#)）已经详细介绍了索引阶段以及全局和局部搜索机制，本文将不再赘述这些细节。然而，**DRIFT搜索**作为一种融合了全局与局部搜索方法的新兴策略，尚未被深入探讨，而这正是本文的重点。DRIFT是一种较新的方法，它结合了全局和局部搜索的特点。该技术首先通过向量搜索利用社区信息来建立一个广泛的查询起点，然后利用这些社区洞察来将原始问题细化为详细的后续查询。这使得DRIFT能够动态遍历知识图谱，以检索关于实体、关系和其他局部细节的特定信息，从而在计算效率与全面的答案质量之间取得平衡。



基于LlamaIndex工作流和Neo4j实现的DRIFT搜索流程。图片由作者绘制。

该实现利用 [LlamaIndex 工作流](#) 来编排DRIFT搜索过程的几个关键步骤。它首先进行 **HyDE生成**，根据样本社区报告创建假设性答案，以优化查询表示。

随后，**社区搜索阶段**通过向量相似度识别最相关的社区报告，为查询提供广泛的上下文。系统分析这些结果以生成初步的中间答案和一组用于深入调查的后续查询。

这些后续查询在**局部搜索阶段**并行执行，从知识图谱中检索目标信息，包括文本块、实体、关系以及额外的社区报告。这个过程可以迭代到最大深度，每一轮都可能产生新的后续查询。

最后，**答案生成阶段**综合了整个过程中收集到的所有中间答案，将广泛的社区层面洞察与详细的局部发现相结合，从而产生一个全面的响应。这种方法平衡了广度与深度，从社区上下文的广阔视角开始，逐步深入到具体细节。

该DRIFT搜索的实现，已针对LlamaIndex工作流和Neo4j进行了适配。其方法是通过对微软GraphRAG代码进行逆向工程而得，因此可能与原始实现存在一些差异。

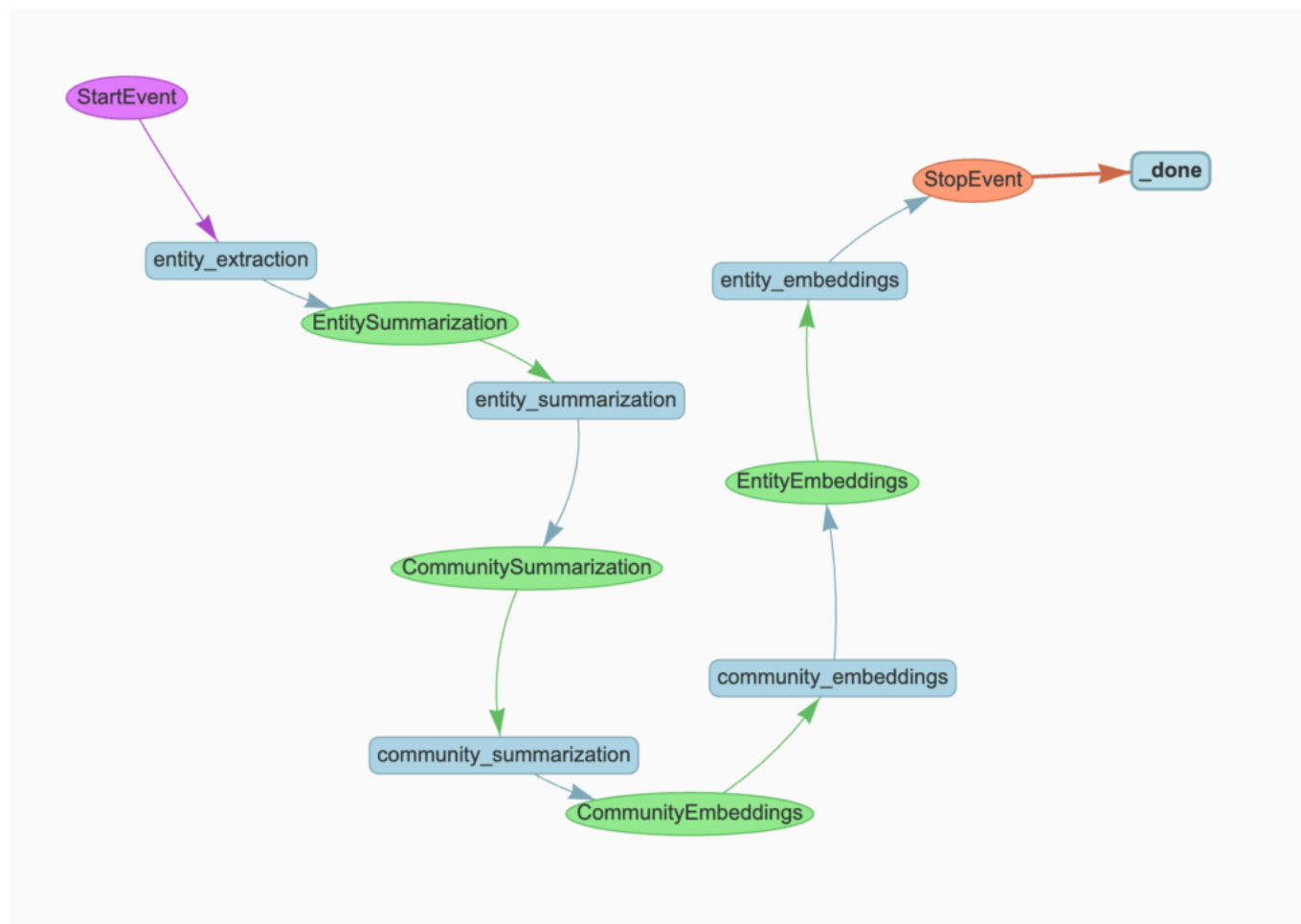
相关代码已在[GitHub](#)上提供。

数据集

为了演示，本文将使用路易斯·卡罗尔（Lewis Carroll）的《爱丽丝梦游仙境》，这是一部可从[古腾堡计划](#)免费获取的经典文本。这个叙事丰富的数据集，其相互关联的人物、地点和事件，使其成为展示 GraphRAG 强大功能的绝佳选择。

数据摄取

在数据摄取过程中，本文将重用为[之前的博客文章](#)开发的[微软 GraphRAG 索引实现](#)，并将其适配到 LlamaIndex 工作流程中。



索引工作流。图片由作者绘制。

摄取管道遵循标准的 GraphRAG 方法，主要包括三个阶段：

```
class MSGraphRAGIngestion(Workflow):  
    @step  
    async def entity_extraction(self, ev: StartEvent) -> EntitySummarization:  
        chunks = splitter.split_text(ev.text)  
        await ms_graph.extract_nodes_and_rels(chunks, ev.allowed_entities)  
        return EntitySummarization()
```

```

@step
async def entity_summarization(
    self, ev: EntitySummarization
) -> CommunitySummarization:
    await ms_graph.summarize_nodes_and_rels()
    return CommunitySummarization()

@step
async def community_summarization(
    self, ev: CommunitySummarization
) -> CommunityEmbeddings:
    await ms_graph.summarize_communities()
    return CommunityEmbeddings()

```

该 workflow 从文本块中提取实体和关系，生成节点和关系的摘要，然后创建分层社区摘要。

摘要完成后，系统会为社区和实体生成向量嵌入，以实现相似度搜索。以下是社区嵌入步骤：

```

@step
async def community_embeddings(self, ev: CommunityEmbeddings) -> EntityEmbeddings:
    # 从图数据库中获取所有社区
    communities = ms_graph.query(
        """
MATCH (c:__Community__)
WHERE c.summary IS NOT NULL AND c.rating > $min_community_rating
RETURN coalesce(c.title, "") + " " + c.summary AS community_description, c.id AS
community_id
""",
        params={"min_community_rating": MIN_COMMUNITY_RATING},
    )
    if communities:
        # 从社区描述生成向量嵌入
        response = await client.embeddings.create(
            input=[c["community_description"] for c in communities],
            model=TEXT_EMBEDDING_MODEL,
        )
        # 在图中存储嵌入并创建向量索引
        embeds = [
            {
                "community_id": community["community_id"],
                "embedding": embedding.embedding,
            }
            for community, embedding in zip(communities, response.data)
        ]

```

```

]

ms_graph.query(
    """UNWIND $data as row

MATCH (c:__Community__ {id: row.community_id})

CALL db.create.setNodeVectorProperty(c, 'embedding', row.embedding)""",
    params={"data": embeds},
)

ms_graph.query(
    "CREATE VECTOR INDEX community IF NOT EXISTS FOR (c:__Community__) ON
c.embedding"
)

return EntityEmbeddings()

```

同样的流程也应用于实体嵌入，创建DRIFT搜索中基于相似度的检索所需的向量索引。

DRIFT搜索详解

DRIFT搜索是一种直观的信息检索方法：首先了解大局，然后在需要时深入研究具体细节。DRIFT不是立即在文档或实体层面搜索精确匹配，而是首先查阅社区摘要，这些摘要是捕获知识图谱中主要主题和话题的高级概述。

一旦DRIFT识别出相关的更高层级信息，它会智能地生成后续查询，以检索关于特定实体、关系和源文档的精确信息。这种两阶段方法模拟了人类自然寻求信息的方式：我们首先通过一般性概述获得方向，然后提出有针对性的问题来填补细节。通过将全局搜索的全面覆盖与局部搜索的精确性相结合，DRIFT在不产生处理所有社区报告或文档的计算开销的情况下，实现了广度和深度的统一。

接下来，将详细介绍实现的每个阶段。

相关代码已在[GitHub](#)上提供。

社区搜索

DRIFT利用HyDE（假设性文档嵌入）来提高向量搜索的准确性。HyDE不是直接嵌入用户的查询，而是首先生成一个假设性答案，然后将其用于相似度搜索。这种方法之所以有效，是因为假设性答案在语义上比原始查询更接近实际的社区摘要。

```

@step
async def hyde_generation(self, ev: StartEvent) -> CommunitySearch:
    # 获取一份随机社区报告作为HyDE生成的模板
    random_community_report = driver.execute_query(
        """
        MATCH (c:__Community__)

```

```

WHERE c.summary IS NOT NULL
RETURN coalesce(c.title, "") + " " + c.summary AS community_description"",
    result_transformer_=lambda r: r.data(),
)
# 生成假设性答案以改善查询表示
hyde = HYDE_PROMPT.format(
    query=ev.query, template=random_community_report[0]["community_description"]
)
hyde_response = await client.responses.create(
    model="gpt-5-mini",
    input=[{"role": "user", "content": hyde}],
    reasoning={"effort": "low"},
)
return CommunitySearch(query=ev.query, hyde_query=hyde_response.output_text)

```

接下来，系统会嵌入HyDE查询，并通过向量相似度检索排名前5的最相关社区报告。然后，它会提示大型语言模型（LLM）根据这些报告生成中间答案，并识别出用于深入调查的后续查询。中间答案会被存储，所有后续查询将并行分派到局部搜索阶段。

```

@step
async def community_search(self, ctx: Context, ev: CommunitySearch) -> LocalSearch:
    # 从HyDE增强的查询创建嵌入
    embedding_response = await client.embeddings.create(
        input=ev.hyde_query, model=TEXT_EMBEDDING_MODEL
    )
    embedding = embedding_response.data[0].embedding

    # 通过向量相似度查找排名前5的最相关社区报告
    community_reports = driver.execute_query(
        """
CALL db.index.vector.queryNodes('community', 5, $embedding) YIELD node, score
RETURN 'community-' + node.id AS source_id, node.summary AS community_summary
""",
        result_transformer_=lambda r: r.data(),
        embedding=embedding,
    )

    # 生成初始答案并识别所需附加信息
    initial_prompt = DRIFT_PRIMER_PROMPT.format(
        query=ev.query, community_reports=community_reports
    )
    initial_response = await client.responses.create(
        model="gpt-5-mini",

```

```

input=[{"role": "user", "content": initial_prompt}],
reasoning={"effort": "low"},
)
response_json = json_repair.loads(initial_response.output_text)
print(f"Initial intermediate response: {response_json['intermediate_answer']}")

# 存储初始答案并准备进行并行局部搜索
async with ctx.store.edit_state() as ctx_state:
    ctx_state["intermediate_answers"] = [
        {
            "intermediate_answer": response_json["intermediate_answer"],
            "score": response_json["score"],
        }
    ]
    ctx_state["local_search_num"] = len(response_json["follow_up_queries"])

# 并行分派后续查询
for local_query in response_json["follow_up_queries"]:
    ctx.send_event(LocalSearch(query=ev.query, local_query=local_query))
return None

```

这确立了DRIFT的核心方法：从HyDE增强的社区搜索开始，进行广泛探索，然后通过有针对性的后续查询深入细节。

局部搜索

局部搜索阶段并行执行后续查询，以深入挖掘具体细节。每个查询通过基于实体的向量搜索检索目标上下文，然后生成中间答案，并可能产生更多的后续查询。

```

@step(num_workers=5)
async def local_search(self, ev: LocalSearch) -> LocalSearchResults:
    print(f"Running local query: {ev.local_query}")

    # 为局部查询创建嵌入
    response = await client.embeddings.create(
        input=ev.local_query, model=TEXT_EMBEDDING_MODEL
    )
    embedding = response.data[0].embedding

    # 检索相关实体并收集其关联上下文：
    # - 提及实体的文本块
    # - 实体所属的社区报告
    # - 检索到的实体之间的关系

```

```

# - 实体描述

local_reports = driver.execute_query(
    """

CALL db.index.vector.queryNodes('entity', 5, $embedding) YIELD node, score
WITH collect(node) AS nodes
WITH
collect {
    UNWIND nodes as n
    MATCH (n)-[:MENTIONS]->(c:__Chunk__)
    WITH c, count(distinct n) as freq
    RETURN {chunkText: c.text, source_id: 'chunk-' + c.id}
    ORDER BY freq DESC
    LIMIT 3
} AS text_mapping,
collect {
    UNWIND nodes as n
    MATCH (n)-[:IN_COMMUNITY*]->(c:__Community__)
    WHERE c.summary IS NOT NULL
    WITH c, c.rating as rank
    RETURN {summary: c.summary, source_id: 'community-' + c.id}
    ORDER BY rank DESC
    LIMIT 3
} AS report_mapping,
collect {
    UNWIND nodes as n
    MATCH (n)-[r:SUMMARIZED_RELATIONSHIP]-(m)
    WHERE m IN nodes
    RETURN {descriptionText: r.summary, source_id: 'relationship-' + n.name + '-' +
m.name}
    LIMIT 3
} as insideRels,
collect {
    UNWIND nodes as n
    RETURN {descriptionText: n.summary, source_id: 'node-' + n.name}
} as entities
RETURN {Chunks: text_mapping, Reports: report_mapping,
    Relationships: insideRels,
    Entities: entities} AS output
""",
    result_transformer=lambda r: r.data(),
    embedding=embedding,
)

# 根据检索到的上下文生成答案

local_prompt = DRIFT_LOCAL_SYSTEM_PROMPT.format(

```



```

        response_type=DEFAULT_RESPONSE_TYPE,
        context_data=local_reports,
        global_query=ev.query,
    )
    local_response = await client.responses.create(
        model="gpt-5-mini",
        input=[{"role": "user", "content": local_prompt}],
        reasoning={"effort": "low"},
    )
    response_json = json_repair.loads(local_response.output_text)

    # 限制后续查询以防止指数级增长
    response_json["follow_up_queries"] = response_json["follow_up_queries"]
[:LOCAL_TOP_K]

    return LocalSearchResults(results=response_json, query=ev.query)

```

下一步编排迭代深化过程。它使用 `collect_events` 等待所有并行搜索完成，然后决定是否继续深入。如果当前深度尚未达到最大值（本文使用最大深度为2），它将从所有结果中提取后续查询，存储中间答案，并分派下一轮并行搜索。

```

@step
async def local_search_results(
    self, ctx: Context, ev: LocalSearchResults
) -> LocalSearch | FinalAnswer:
    local_search_num = await ctx.store.get("local_search_num")

    # 等待所有并行搜索完成
    results = ctx.collect_events(ev, [LocalSearchResults] * local_search_num)
    if results is None:
        return None

    intermediate_results = [
        {
            "intermediate_answer": event.results["response"],
            "score": event.results["score"],
        }
        for event in results
    ]
    current_depth = await ctx.store.get("local_search_depth", default=1)
    query = [ev.query for ev in results][0]

    # 如果尚未达到最大深度，则继续深入

```

```

if current_depth < MAX_LOCAL_SEARCH_DEPTH:
    await ctx.store.set("local_search_depth", current_depth + 1)
    follow_up_queries = [
        query
        for event in results
        for query in event.results["follow_up_queries"]
    ]

    # 存储中间答案并分派下一轮搜索
    async with ctx.store.edit_state() as ctx_state:
        ctx_state["intermediate_answers"].extend(intermediate_results)
        ctx_state["local_search_num"] = len(follow_up_queries)

    for local_query in follow_up_queries:
        ctx.send_event(LocalSearch(query=query, local_query=local_query))
    return None
else:
    return FinalAnswer(query=query)

```

这创建了一个迭代细化循环，其中每个深度级别都建立在先前的发现之上。一旦达到最大深度，它将触发最终答案的生成。

最终答案生成

最后一步是将DRIFT搜索过程中收集到的所有中间答案综合为一个全面的响应。这包括社区搜索的初始答案以及局部搜索迭代中生成的所有答案。

```

@step
async def final_answer_generation(self, ctx: Context, ev: FinalAnswer) -> StopEvent:
    # 检索搜索过程中收集到的所有中间答案
    intermediate_answers = await ctx.store.get("intermediate_answers")

    # 将所有发现综合为一个全面的最终响应
    answer_prompt = DRIFT_REDUCE_PROMPT.format(
        response_type=DEFAULT_RESPONSE_TYPE,
        context_data=intermediate_answers,
        global_query=ev.query,
    )

    answer_response = await client.responses.create(
        model="gpt-5-mini",
        input=[
            {"role": "developer", "content": answer_prompt},
            {"role": "user", "content": ev.query},

```

```
    ],  
    reasoning={"effort": "low"},  
)  
  
return StopEvent(result=answer_response.output_text)
```

总结与展望

DRIFT搜索提出了一种平衡全局搜索广度与局部搜索精度有趣的策略。通过从社区级上下文开始，并通过迭代的后续查询逐步深入，它避免了处理所有社区报告的计算开销，同时仍保持了全面的覆盖。

然而，仍有几个方面可以改进。当前的实现平等对待所有中间答案，但根据其置信度分数进行过滤可以提高最终答案的质量并减少噪音。同样，后续查询可以在执行前根据其相关性或潜在信息增益进行排序，确保首先追溯最有前景的线索。

另一个有前景的增强是引入查询细化步骤，利用大型语言模型（LLM）分析所有生成的后续查询，对相似的查询进行分组以避免冗余搜索，并过滤掉不太可能产生有用信息的查询。这可以在保持答案质量的同时，显著减少局部搜索的数量。